You said:

codigo(2).py
Code

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 1m 36s

ChatGPT said:

Veredito: MALWARE — alta confiança

O arquivo `codigo(2).py` é **malicioso**. A análise estática revela um carregador de malware (*loader/downloader*) que utiliza ofuscação, consulta a blockchain Solana para obter instruções de comando e controle (C2), baixa componentes externos e executa código JavaScript remotamente. O endereço Solana encontrado também aparece em relatórios públicos sobre campanhas de malware. Corgea +1

1. Análise passo a passo

Etapa 1 — Identificação da ofuscação

O arquivo original possui apenas nove linhas, mas esconde um programa muito maior dentro de uma longa sequência codificada.

O código utiliza três técnicas:

- **Base64:** codifica o conteúdo oculto.
- **Zlib:** descompacta os dados codificados.
- **XOR com a constante 134 (`0x86`):** transforma os bytes para recuperar o código original.

Finalmente, utiliza `compile()` e `exec()` para executar o conteúdo recuperado diretamente na memória. codigo.py

Avaliação: a combinação de ofuscação em camadas com execução dinâmica é suspeita e dificulta a análise por ferramentas simples.

Etapa 2 — Recuperação do código oculto

Decodifiquei o conteúdo sem executar o programa.

O resultado contém aproximadamente **15,5 KB de código Python**, que importa módulos capazes de realizar operações de rede, manipular arquivos, baixar recursos, iniciar processos e executar tarefas assíncronas.

Entre os elementos relevantes estão `urllib.request`, `subprocess`, `base64`, `asyncio` e `zipfile`.

Avaliação: essas bibliotecas também podem ser utilizadas em programas legítimos, mas as operações realizadas por esse código, consideradas em conjunto, revelam comportamento malicioso.

Etapa 3 — Verificação da localização do sistema

O programa aguarda dez segundos e executa a função `_isRussianSystem()`.

Ela verifica o idioma, as variáveis de ambiente, a localidade e informações sobre o fuso horário do sistema. Quando identifica determinadas combinações associadas à Rússia, o programa interrompe sua execução.

Avaliação: esse comportamento é compatível com *geofencing*, uma técnica usada por operadores de malware para evitar a execução em determinadas regiões ou ambientes de análise.

Etapa 4 — Comunicação de comando e controle pela blockchain

O código consulta diversos serviços RPC da rede Solana para obter transações associadas ao seguinte endereço:

```
BjVeAjPrSKFiingBn4vZvghsGj9KCE8AJVtbc9S8o8SC
```

A função `_getSignFAddress()` pesquisa assinaturas de transações. Em seguida, a função `fvzsrnzlh()` procura transações que contenham um campo `memo`, extrai seu conteúdo e tenta interpretá-lo como JSON.

Esse JSON pode conter o campo `link`, utilizado para localizar o próximo componente.

A mesma carteira foi documentada em relatórios públicos sobre campanhas maliciosas que utilizam a blockchain para distribuir instruções e localizar cargas adicionais.

Corqea +1

Avaliação: há evidências fortes de utilização de infraestrutura de comando e controle baseada em blockchain, uma técnica que pode dificultar o bloqueio da infraestrutura do atacante.

Etapa 5 — Download de componentes externos

Após obter as instruções, o programa faz uma requisição HTTP para a URL recuperada. Ele obtém o conteúdo da resposta e também lê cabeçalhos chamados `ivbase64` e `secretkey`.

Esses valores são posteriormente incorporados ao código JavaScript gerado pelo carregador.

O programa também baixa o Node.js, versão 22.9.0, caso não encontre uma instalação no diretório pessoal do usuário. O arquivo baixado é extraído para esse diretório.

Avaliação: o programa prepara um ambiente de execução para componentes externos que não estão integralmente presentes no arquivo original.

Etapa 6 — Execução de código remoto

Esta é uma das evidências mais graves.

O código construído contém a instrução:

```
eval(atob('...'));
```

O valor representado por `...` é substituído pelo conteúdo remoto obtido pelo programa. A função `atob()` decodifica esse conteúdo em Base64 e `eval()` tenta executá-lo como JavaScript.

O carregador grava o código em um arquivo chamado `i.js` e inicia o Node.js para executá-lo, redirecionando as saídas padrão para ocultar mensagens do processo.

Avaliação: isso cria um mecanismo para executar código recebido de uma infraestrutura externa. As ações realizadas pelo JavaScript dependem da carga adicional obtida durante a execução.

Etapa 7 — Controle de execuções repetidas

O código consulta o arquivo `init.json` no diretório pessoal do usuário. Se encontrar uma data ainda válida dentro do período de dois dias previsto pelo programa, pode interromper a execução.

Em determinadas condições, ele também atualiza ou cria esse arquivo.

Isso funciona como um mecanismo de controle de execuções repetidas. Por si só, não comprova persistência permanente no sistema.

Avaliação: o comportamento complementa o mecanismo de carregamento e pode reduzir a frequência de execução.

2. Principais indicadores encontrados

Indicador	Interpretação
Base64 + Zlib + XOR	Ofuscação do código
`exec(compile(...))`	Execução do conteúdo oculto
Filtro de sistemas russos	Geofencing
Consulta a transações Solana	Obtenção de instruções de C2
Download do Node.js	Preparação do ambiente de execução
JavaScript remoto com `eval()`	Execução dinâmica de código externo
Arquivos `i.js` e `init.json`	Artefatos criados durante a execução

A presença conjunta desses indicadores, especialmente a resolução de instruções pela blockchain e a execução de código remoto, sustenta a classificação maliciosa.

3. Qual é a categoria principal do malware?

Loader / Downloader de malware

Categoria principal · Confiança alta

Sua função observável é recuperar instruções de uma infraestrutura externa, baixar componentes e iniciar uma carga JavaScript em outra etapa da execução. A comunicação de C2 utiliza a blockchain Solana.

Há associação dos indicadores de infraestrutura com campanhas descritas publicamente como GlassWorm/ForceMemo. Algumas cargas dessas campanhas foram relacionadas ao roubo de credenciais, tokens e segredos. Entretanto, **não é possível afirmar, apenas com este arquivo, que a carga final específica seja um infostealer**, pois o JavaScript entregue pelo servidor não foi obtido e analisado nesta inspeção. Corqea +1

4. Conclusão final

Classificação: MALWARE.

Tipo principal: carregador e downloader de segunda etapa, com execução remota de JavaScript e infraestrutura de comando e controle baseada em Solana.

A conclusão se baseia nos comportamentos encontrados no código recuperado e nos indicadores públicos correspondentes. Não executei o arquivo nem acessei sua infraestrutura de comando e controle durante a análise.

O SHA-256 do arquivo original é:

```
d29d46edd83c523a817a693748cc462c405f093aece2e2e533673e24f98d6dc1
```

Recomendação: não execute o arquivo. Caso ele já tenha sido executado, trate o sistema como potencialmente comprometido, isole-o da rede e avalie a necessidade de trocar credenciais e revogar tokens a partir de um dispositivo confiável.

Também posso criar um ↳ diagrama do fluxo do malware.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal deste malware é **Loader/Downloader (carregador e baixador de malware)**.

Ele atua como um **carregador de segunda etapa**, utilizando a blockchain Solana para obter instruções de comando e controle (C2), baixar componentes externos e executar código JavaScript remotamente.

Categoria secundária possível: *Infostealer* (ladrão de informações), caso a carga final baixada realize roubo de credenciais ou tokens — algo que não foi confirmado diretamente neste arquivo.